

BÀI TẬP LỚN CÁC NHÓM

(Dự kiến nội dung phục vụ lấy điểm giữa kỳ và cuối kỳ)

1. Cơ cấu nhóm và mối liên hệ giữa các đề tài

Nhóm	Đề tài	Quy mô	Đầu ra kết nối
1	Math Data Lake - ingestion/schema	7	Snapshot và data contract
2	Math Data Quality - dedup/leakage	6	Clean splits và quality report
3	Lean Retrieval - sparse/hybrid	6	Keyword/structural search API
4	Lean Retrieval - dense/ANN	6	Vector search API và Pareto report
5	Autoformalization Pipeline	6	Candidate Lean + error logs
6	Verifier-Guided Repair	6	Repaired Lean + error taxonomy
7	Proof-Search Analytics	6	Trace lake + pruning benchmark
8	AI Math Tutor	6	Streaming learner state + verified hints

Nhóm 1 có 7 bạn vì cần xử lý nhiều nguồn và điều phối data contract. Các nhóm liên quan phải thống nhất interface ở tuần 2, nhưng mỗi nhóm vẫn phải có snapshot hoặc mock input để không phụ thuộc hoàn toàn vào tiến độ của nhóm khác.

2. Yêu cầu chung bắt buộc

- Câu hỏi nghiên cứu phải chỉ ra nút thắt Big Data: khối lượng, đa dạng, tốc độ, chất lượng, chi phí tính toán, độ trễ hoặc khả năng chịu tải.
- Có baseline đơn giản, chạy end-to-end trước khi triển khai phương pháp nâng cao.
- Có tối thiểu ba mức quy mô hoặc ba mức tải; ưu tiên khoảng cách gần một bậc độ lớn nếu tài nguyên cho phép.
- Báo cáo đồng thời chất lượng và chi phí; không chọn riêng metric có lợi cho phương pháp của nhóm.
- Mọi biểu đồ phải được sinh từ raw results bằng script; ghi cấu hình phần cứng, phần mềm, seed, timeout và số lần lặp.
- Có error analysis, ablation và limitation; không chỉ báo cáo kết quả tốt nhất.
- Code, dữ liệu mẫu, data/model card và README phải cho phép một thành viên khác chạy lại.
- Mỗi thành viên có vai trò chính và dự phòng; khi bảo vệ phải hiểu toàn bộ pipeline.
- Việc sử dụng AI phải được khai báo; mọi code và kết luận do AI hỗ trợ vẫn phải được nhóm kiểm chứng.

3. Tiến độ chung 10 tuần

Tuần	Nhiệm vụ bắt buộc	Mình chứng cần nộp
1	Chốt nhóm, câu hỏi, nguồn dữ liệu, baseline, metric, phân vai và rủi ro.	Đề cương 1 trang + board nhiệm vụ.
2	Data contract, snapshot/mock input, schema và script đọc dữ liệu.	Data sample + data card v0.1.
3	Baseline chạy end-to-end trên dữ liệu nhỏ.	Code + raw metric + demo nội bộ.
4	Kiểm thử, checkpoint và sửa đường dữ liệu.	Checkpoint report; không tối ưu nếu baseline chưa ổn.

Tuần	Nhiệm vụ bắt buộc	Minh chứng cần nộp
5–6	Phiên bản phân tán/serving, logging, chịu lỗi và reproducibility.	Pipeline v0.2 + test + environment lock.
7	Chạy thí nghiệm scale và tải; lặp ít nhất 3 lần khi phù hợp.	Raw CSV/JSON + cấu hình từng run.
8	Ablation, error analysis, fairness/privacy nếu liên quan.	Bảng ablation + taxonomy lỗi + limitations.
9	Đóng gói sản phẩm và báo cáo dạng bài báo ngắn.	Release candidate + report + demo script.
10	Demo, phản biện, bảo vệ cá nhân và bàn giao.	Final release + contribution log.

4. Cấu trúc sản phẩm bàn giao

- README.md: mục tiêu, kiến trúc, cài đặt, dữ liệu, lệnh chạy nhanh và cách tái tạo kết quả chính.
- src/: mã nguồn có cấu trúc; tests/: kiểm thử; configs/: cấu hình từng thí nghiệm.
- data_sample/: dữ liệu mẫu hợp pháp; scripts/: tải hoặc tạo dữ liệu đầy đủ.
- results/raw/: kết quả thô; results/figures/: bảng/hình được tạo bằng script.
- docs/: data card, model/system card, AI-use statement, privacy/license note.
- report.pdf/docx: báo cáo 8-12 trang; slides/demo: slide và kịch bản trình diễn.
- contributions.md: vai trò, issue, pull request và phần bảo vệ của từng thành viên.

5. Rubric đánh giá sản phẩm bài tập lớn

Thành phần	Tỷ trọng gợi ý	Điều kiện đạt tốt
Câu hỏi và vai trò Big Data	10%	Nút thắt rõ, giả thuyết đo được, phạm vi phù hợp.
Dữ liệu và provenance	15%	Nguồn/giấy phép/version rõ; schema và quality checks.
Kiến trúc và triển khai	20%	Pipeline chạy ổn định, test, logging, interface rõ.
Baseline	10%	Đơn giản, công bằng, tái lập, chạy trước phương pháp chính.
Scalability experiments	20%	≥ 3 mức scale/tải; raw results; cấu hình và số lần lặp.
Đánh giá và phân tích	10%	Metric phù hợp; ablation, error analysis, trade-off.
Bàn giao và tái lập	10%	README, environment, data sample, scripts chạy lại.
Trình bày/đóng góp cá nhân	5%	Demo rõ; trả lời được toàn pipeline và phần phụ trách.

LƯU Ý VỀ ĐIỂM HỌC PHẦN

Cơ cấu điểm toàn học phần vẫn tuân theo đề cương INT4418. Rubric trên dùng để đánh giá chất lượng sản phẩm và làm bằng chứng cho phần bài tập, seminar hoặc vấn đáp theo cách tổ chức chính thức của lớp.

NHÓM 1. Math Data Lake - Thu nhận dữ liệu và lược đồ thống nhất

TÓM TẮT ĐỀ BÀI

Quy mô nhóm: 7 bạn. Xây kho dữ liệu toán học nhiều nguồn, có cấu trúc thống nhất, truy vấn được và truy vết được nguồn gốc. Nguồn khởi đầu: [1], [2], [3], [4], [5]; xem Phụ lục D.

1. Mục tiêu và câu hỏi nghiên cứu

Xây kho dữ liệu toán học nhiều nguồn, có cấu trúc thống nhất, truy vấn được và truy vết được nguồn gốc.

1. Có thể hợp nhất dữ liệu văn bản, LaTeX và Lean vào một lược đồ chung mà không làm mất thông tin nguồn gốc hay không?
2. Khi dữ liệu tăng từ 10 nghìn lên hàng trăm nghìn hoặc toàn bộ kho, định dạng lưu trữ và cách chia vùng ảnh hưởng thế nào tới thời gian, bộ nhớ và dung lượng?

Vai trò của Dữ liệu lớn: Thể hiện rõ ở tính đa dạng nguồn và biểu diễn, khối lượng hàng trăm nghìn bài toán, chi phí đọc–ghi, số lượng tệp nhỏ, chia vùng dữ liệu và khả năng xử lý lại khi dữ liệu cập nhật.

2. Dữ liệu thực tế

- NuminaMath-CoT: khoảng 860 nghìn bài toán và lời giải theo chuỗi suy luận. Dùng cho dữ liệu văn bản/LaTeX quy mô lớn; phải kiểm tra trường nguồn và giấy phép tại thời điểm tải.
- ProofNet: 371 ví dụ toán đại học gồm phát biểu tự nhiên, chứng minh tự nhiên và phát biểu hình thức Lean 3. Dùng như nguồn nhỏ nhưng giàu cấu trúc để kiểm tra lược đồ.
- mathlib4/LeanDojo Benchmark 4: dữ liệu định lý, chứng minh, tactic và premise từ thư viện Lean 4. Chỉ lấy snapshot cố định, ghi rõ commit hoặc phiên bản.

YÊU CẦU DỮ LIỆU

Không tải và đưa toàn bộ dữ liệu lớn vào Git. Chỉ lưu sample hợp pháp, manifest, checksum và script tải/tạo dữ liệu. Mọi split phải cố định và được version hóa.

3. Kiến trúc gợi ý

3. Vùng raw: giữ nguyên tệp tải về, checksum, ngày tải và giấy phép.
4. Vùng staging: chuyển mã hóa, chuẩn hóa tên trường, làm sạch lỗi cơ bản.
5. Spark ETL: đọc nhiều nguồn, ánh xạ sang lược đồ chung và ghi thống kê lỗi.
6. Vùng curated: lưu Parquet, chia vùng theo nguồn/phiên bản/chủ đề; Iceberg là phần mở rộng nếu nhóm đủ năng lực.
7. Catalog và data card: mô tả bảng, cột, nguồn, phiên bản, giới hạn và ví dụ truy vấn.

Nhóm có thể thay kiến trúc nếu giải thích được tác động tới tính đúng, chi phí, khả năng mở rộng và khả năng tích hợp.

4. Công cụ gợi ý

- Python
- PySpark/Apache Spark
- Parquet
- DuckDB để kiểm tra cục bộ
- Apache Iceberg (không bắt buộc)
- Git/DVC hoặc manifest phiên bản

5. Phân vai

Vai trò	Trách nhiệm chính
Thành viên 1	Điều phối và data contract
Thành viên 2	Thu nhận NuminaMath
Thành viên 3	Thu nhận ProofNet
Thành viên 4	Thu nhận mathlib4/LeanDojo
Thành viên 5	Spark ETL
Thành viên 6	Lưu trữ và benchmark
Thành viên 7	QA, data card và tích hợp

Nhóm 7 bạn có thể bố trí đúng bảy vai trò trên. Các nhóm 6 bạn có thể gộp hai vai trò gần nhau nhưng phải có người dự phòng.

6. Nhiệm vụ từng tuần

Tuần	Nhiệm vụ bắt buộc	Minh chứng cần nộp
1	Khảo sát nguồn, giấy phép, trường dữ liệu; chốt câu hỏi và lược đồ bản nháp.	Đề cương 1 trang; danh mục nguồn; sơ đồ lược đồ v0.1.
2	Tải snapshot nhỏ; xây manifest và data contract; tạo 100–1.000 bản ghi mẫu.	Raw sample; checksum; data contract; script đọc được cả ba nguồn.
3	Cài baseline JSONL + Python tuần tự; đo thời gian và dung lượng.	Baseline chạy end-to-end; raw timing; báo cáo lỗi đọc.
4	Cài Spark ETL; chuẩn hóa text, LaTeX, Lean và metadata; thêm test.	Pipeline Spark v0.1; kiểm thử lược đồ; báo cáo tỷ lệ lỗi.
5	Thiết kế Parquet, partition và kích thước tệp; xử lý small-file problem.	Ba cấu hình lưu trữ; bảng số tệp, dung lượng, thời gian đọc.
6	Bổ sung provenance, catalog và cơ chế cập nhật gia tăng.	Data card; catalog; thử nghiệm thêm một batch dữ liệu mới.
7	Chạy 10K, 100K và toàn bộ dữ liệu; lặp ít nhất 3 lần.	CSV kết quả thô; biểu đồ runtime, throughput, RAM, bytes I/O.
8	Ablation: JSONL/Parquet, có/không partition, nhiều tệp nhỏ/tệp hợp lý.	Bảng ablation; phân tích điểm nghẽn và trường hợp lỗi.
9	Đóng gói kho mẫu, mã, truy vấn và báo cáo dạng bài báo ngắn.	Release candidate; README; bản báo cáo; demo script.
10	Tích hợp API dữ liệu cho nhóm khác; demo và bảo vệ cá nhân.	Bản bàn giao cuối; video/demo; bảng đóng góp cá nhân.

7. Baseline bắt buộc

JSONL thô + chương trình Python tuần tự trên một máy; không chia vùng và không tối ưu định dạng.

QUY TẮC BASELINE

Baseline phải dùng cùng dữ liệu, cùng split, cùng phần cứng và cùng timeout với phương pháp chính. Không thay đổi điều kiện để làm phương pháp mới trông tốt hơn.

8. Thí nghiệm khả năng mở rộng

- Quy mô dữ liệu: 10K → 100K → toàn bộ snapshot.
- Định dạng: JSONL/CSV so với Parquet.
- Chia vùng: không partition, partition theo nguồn, partition theo nguồn + phiên bản.
- Kích thước tệp mục tiêu: nhiều tệp nhỏ so với tệp đã gộp.
- Số worker/partition Spark: ít, trung bình, cao; giải thích overhead.

Mỗi cấu hình cần lưu: timestamp, commit, data version, machine, software version, seed, parameters, runtime, status và đường dẫn log. Nếu runtime biến động, chạy ít nhất ba lần và báo cáo median cùng khoảng biến thiên.

9. Chỉ số đánh giá

Nhóm chỉ số	Các chỉ số phải xem xét
Chất lượng	Runtime, Throughput bản ghi/giây, Peak RAM
Hiệu năng/chi phí	Dung lượng lưu, Bytes đọc/ghi, Tỷ lệ lỗi lược đồ, Thời gian truy vấn mẫu

Không phải mọi chỉ số đều dùng làm chỉ số chính. Nhóm phải chọn 1–2 primary metrics, các chỉ số còn lại dùng để giải thích trade-off và giới hạn.

10. Sản phẩm bàn giao

- Kho Parquet mẫu và manifest
- Lược đồ + data dictionary
- Pipeline ingest/ETL
- Data card và license note
- Bộ truy vấn kiểm tra
- Raw results + biểu đồ
- README và demo

11. Tiêu chí nghiệm thu tối thiểu

- Đọc được tối thiểu 3 nguồn
- Tái tạo được curated sample bằng một lệnh
- Có baseline và ít nhất 3 mức quy mô
- Không mất trường nguồn/giấy phép/phiên bản
- Một nhóm khác sử dụng được data contract

12. Khả năng phát triển thành bài báo

Có thể phát triển thành data paper về kho dữ liệu toán đa biểu diễn, hoặc bài về thiết kế lược đồ/provenance và benchmark thu nhận dữ liệu toán học quy mô lớn.

Để đi tiếp theo hướng bài báo, nhóm phải xác định đóng góp mới so với công trình liên quan, bổ sung baseline mạnh, dùng split công bằng, chạy đủ lặp/ablation và trình bày threat to validity — các nguy cơ làm kết luận thiếu tin cậy.

NHÓM 2. Math Data Quality - Khử trùng lặp và kiểm toán rõ ràng dữ liệu

TÓM TẮT ĐỀ BÀI

Quy mô nhóm: 6 bạn. Xây quy trình đo và cải thiện chất lượng dữ liệu toán, tập trung vào lỗi định dạng, bản ghi trùng/gần trùng và rõ ràng giữa các tập dữ liệu. Nguồn khởi đầu: [1], [2], [5], [6]; xem Phụ lục D.

1. Mục tiêu và câu hỏi nghiên cứu

Xây quy trình đo và cải thiện chất lượng dữ liệu toán, tập trung vào lỗi định dạng, bản ghi trùng/gần trùng và rõ ràng giữa các tập dữ liệu.

- Các phương pháp băm chính xác, MinHash/LSH và embedding khác nhau thế nào về chất lượng và chi phí phát hiện trùng?
- Rõ ràng dữ liệu làm thay đổi kết quả đánh giá mô hình hoặc hệ thống truy hồi ở mức nào?

Vai trò của Dữ liệu lớn: So sánh mọi cặp có chi phí bậc hai; với một triệu bản ghi có thể có gần 10^{12} cặp. Nhóm phải tạo ứng viên hiệu quả, xử lý phân tán và đo trade-off giữa recall, thời gian và bộ nhớ.

2. Dữ liệu thực tế

- Snapshot từ Nhóm 1: text, LaTeX, Lean, nguồn và split.
- NuminaMath-CoT: dùng để kiểm tra trùng câu hỏi/lời giải và dấu hiệu trùng nguồn.
- ProofNet hoặc tập nhỏ được gán nhãn thủ công: dùng làm gold set cho exact duplicate, near duplicate và non-duplicate.

YÊU CẦU DỮ LIỆU

Không tải và đưa toàn bộ dữ liệu lớn vào Git. Chỉ lưu sample hợp pháp, manifest, checksum và script tải/tạo dữ liệu. Mọi split phải cố định và được version hóa.

3. Kiến trúc gợi ý

- Profiling và rule checks: độ dài, ký tự hỏng, trường rỗng, LaTeX lỗi.
- Chuẩn hóa có kiểm soát: khoảng trắng, Unicode, ký hiệu; lưu cả raw và normalized.
- Exact hash để tìm bản sao hoàn toàn giống.
- MinHash/LSH để tạo candidate pairs gần trùng mà không so sánh toàn bộ cặp.
- Pair verifier và manual audit; sau đó tạo clean split và leakage report.

Nhóm có thể thay kiến trúc nếu giải thích được tác động tới tính đúng, chi phí, khả năng mở rộng và khả năng tích hợp.

4. Công cụ gợi ý

- PySpark
- Spark ML MinHashLSH
- Python regex
- RapidFuzz hoặc similarity tự cài
- DuckDB
- Great Expectations (không bắt buộc)

5. Phân vai

Vai trò	Trách nhiệm chính
Thành viên 1	Điều phối/data contract

Vai trò	Trách nhiệm chính
Thành viên 2	Rule-based quality
Thành viên 3	Normalization
Thành viên 4	Exact/near dedup
Thành viên 5	Leakage audit
Thành viên 6	Evaluation và dashboard

Sáu vai trò tương ứng sáu người chính. Mỗi vai trò cần một người khác có thể chạy lại và giải thích khi bảo vệ.

6. Nhiệm vụ từng tuần

Tuần	Nhiệm vụ bắt buộc	Minh chứng cần nộp
1	Định nghĩa taxonomy lỗi và near-duplicate; chốt ngưỡng và câu hỏi.	Tài liệu định nghĩa; 30 ví dụ cho mỗi loại lỗi.
2	Nhận snapshot; tạo gold set 300–500 cặp bằng hai người gán nhãn.	Gold set; hướng dẫn gán nhãn; thống kê độ đồng thuận.
3	Baseline exact hash + regex; tạo báo cáo profiling.	Baseline; quality profile; danh sách lỗi ưu tiên.
4	Cài normalization và MinHash/LSH candidate generation.	Pipeline v0.1; test; số cặp trước/sau tạo ứng viên.
5	Xây pair verifier; tinh chỉnh ngưỡng trên validation set.	Precision/recall theo ngưỡng; error samples.
6	Kiểm toán leakage giữa train/dev/test và giữa các nguồn.	Leakage matrix; clean split v0.1; audit report.
7	Chạy scale 10K, 100K, 1M; lập và lưu raw results.	Runtime, RAM, candidate reduction, precision/recall.
8	Đánh giá ảnh hưởng của làm sạch lên một baseline downstream.	Kết quả trước/sau dedup; phân tích giới hạn.
9	Đóng gói rules, pipeline, dashboard và báo cáo.	Release candidate; README; report; demo.
10	Bàn giao clean split; demo và bảo vệ cá nhân.	Final package; contribution log; presentation.

7. Baseline bắt buộc

Chuẩn hóa tối thiểu + exact hash để tìm bản ghi giống hệt; regex để bắt lỗi định dạng.

QUY TẮC BASELINE

Baseline phải dùng cùng dữ liệu, cùng split, cùng phần cứng và cùng timeout với phương pháp chính. Không thay đổi điều kiện để làm phương pháp mới trông tốt hơn.

8. Thí nghiệm khả năng mở rộng

- 10K $\rightarrow$ 100K $\rightarrow$ 1M bản ghi/cập ứng viên
- Exact hash so với MinHash/LSH
- Các ngưỡng similarity
- Số hash table/bucket
- Trước và sau làm sạch trên một nhiệm vụ downstream

Mỗi cấu hình cần lưu: timestamp, commit, data version, machine, software version, seed, parameters, runtime, status và đường dẫn log. Nếu runtime biến động, chạy ít nhất ba lần và báo cáo median cùng khoảng biến thiên.

9. Chỉ số đánh giá

Nhóm chỉ số	Các chỉ số phải xem xét
Chất lượng	Precision, Recall, F1, Leakage rate
Hiệu năng/chi phí	Candidate reduction ratio, Runtime, Peak RAM, Ảnh hưởng tới metric downstream

Không phải mọi chỉ số đều dùng làm chỉ số chính. Nhóm phải chọn 1–2 primary metrics, các chỉ số còn lại dùng để giải thích trade-off và giới hạn.

10. Sản phẩm bàn giao

- Taxonomy chất lượng
- Gold set
- Rule suite
- Dedup/leakage pipeline
- Clean splits
- Quality dashboard
- Audit report
- Raw results + README

11. Tiêu chí nghiệm thu tối thiểu

- Có gold set và quy trình gán nhãn
- Báo cáo precision/recall chứ không chỉ số lượng trùng
- Không làm mất raw data
- Có clean split và leakage matrix
- Có thí nghiệm downstream hoặc phân tích tác động

12. Khả năng phát triển thành bài báo

Có thể phát triển thành bài về ảnh hưởng của near-duplicate và leakage tới đánh giá AI4Math, hoặc phương pháp khử trùng lặp toán học có khả năng mở rộng.

Để đi tiếp theo hướng bài báo, nhóm phải xác định đóng góp mới so với công trình liên quan, bổ sung baseline mạnh, dùng split công bằng, chạy đủ lặp/ablation và trình bày threat to validity — các nguy cơ làm kết luận thiếu tin cậy.

NHÓM 3. Lean Premise Retrieval - Truy hồi thừa và truy hồi lai

TÓM TẮT ĐỀ BÀI

Quy mô nhóm: 6 bạn. Tìm các định lý/bổ đề phù hợp trong mathlib4 bằng từ khóa, ký hiệu và đặc trưng cấu trúc trước khi gọi bộ chứng minh. Nguồn khởi đầu: [3], [4], [5], [7]; xem Phụ lục D.

1. Mục tiêu và câu hỏi nghiên cứu

Tìm các định lý/bổ đề phù hợp trong mathlib4 bằng từ khóa, ký hiệu và đặc trưng cấu trúc trước khi gọi bộ chứng minh.

15. BM25 có đủ mạnh cho dữ liệu Lean hay cần thêm trường ký hiệu, namespace và dependency?
16. Kết hợp sparse score với tín hiệu cấu trúc có cải thiện Recall@k mà vẫn giữ độ trễ thấp không?

Vai trò của Dữ liệu lớn: Kho định lý lớn làm truy vấn tuyến tính tốn kém; hệ thống phải xây chỉ mục, phục vụ nhiều truy vấn và đo chất lượng–độ trễ–kích thước chỉ mục khi kho tăng.

2. Dữ liệu thực tế

- LeanDojo Benchmark 4: theorem, proof, tactic và premise từ mathlib4.
- mathlib4 source/declarations: tên đầy đủ, namespace, statement và dependency.
- Cùng split truy vấn–premise với Nhóm 4 để so sánh công bằng.

YÊU CẦU DỮ LIỆU

Không tải và đưa toàn bộ dữ liệu lớn vào Git. Chỉ lưu sample hợp pháp, manifest, checksum và script tải/tạo dữ liệu. Mọi split phải cố định và được version hóa.

3. Kiến trúc gợi ý

17. Extractor → document fields
18. Tokenizer giữ từ và ký hiệu Lean
19. Inverted index
20. BM25 baseline
21. Structural features/field weights
22. Hybrid reranker
23. REST API hoặc CLI truy vấn

Nhóm có thể thay kiến trúc nếu giải thích được tác động tới tính đúng, chi phí, khả năng mở rộng và khả năng tích hợp.

4. Công cụ gợi ý

- LeanDojo v2/current benchmark
- PySpark
- OpenSearch/Lucene hoặc rank_bm25
- FastAPI (không bắt buộc)
- DuckDB/Parquet

5. Phân vai

Vai trò	Trách nhiệm chính
Thành viên 1	Data/extraction

Vai trò	Trách nhiệm chính
Thành viên 2	Indexing
Thành viên 3	BM25 baseline
Thành viên 4	Structural features
Thành viên 5	Serving/performance
Thành viên 6	Evaluation/error analysis

Sáu vai trò tương ứng sáu người chính. Mỗi vai trò cần một người khác có thể chạy lại và giải thích khi bảo vệ.

6. Nhiệm vụ từng tuần

Tuần	Nhiệm vụ bắt buộc	Minh chứng cần nộp
1	Định nghĩa truy vấn, premise đúng và split; chốt data contract chung với Nhóm 4.	Benchmark specification; 20 truy vấn kiểm tra.
2	Trích xuất snapshot nhỏ và các trường Lean; tạo index sample.	Parquet sample; data dictionary; query set.
3	Cài TF-IDF/BM25 baseline; đánh giá Recall@k và MRR.	Baseline end-to-end; raw ranking results.
4	Thêm tokenizer ký hiệu, field weighting và namespace.	Các biến thể retrieval; unit tests.
5	Thêm đặc trưng dependency hoặc hybrid reranking.	Ablation plan; model v0.1; latency profile.
6	Đóng gói search service; cache và batch query nếu cần.	API/CLI; load test nhỏ; index manifest.
7	Chạy quy mô 10K, 50K, toàn bộ declarations và tải đồng thời.	Recall/MRR/P95/index size/QPS; raw data.
8	Ablation và error analysis theo chủ đề, ký hiệu và độ dài.	Bảng ablation; 30 failure cases được phân loại.
9	So sánh với Nhóm 4 trên cùng split; viết báo cáo.	Comparison table; report; reproducible scripts.
10	Demo truy hồi và, nếu khả thi, thử với Lean prover.	Final service/index; demo; oral defense.

7. Baseline bắt buộc

TF-IDF hoặc BM25 mặc định trên statement đã tách từ; không dùng embedding và không dùng dependency graph.

QUY TẮC BASELINE

Baseline phải dùng cùng dữ liệu, cùng split, cùng phần cứng và cùng timeout với phương pháp chính. Không thay đổi điều kiện để làm phương pháp mới trông tốt hơn.

8. Thí nghiệm khả năng mở rộng

- Index 10K → 50K → toàn bộ
- BM25 mặc định vs field-aware
- Có/không ký hiệu Lean
- Có/không structural reranking
- 1, 10, 50 truy vấn đồng thời

Mỗi cấu hình cần lưu: timestamp, commit, data version, machine, software version, seed, parameters, runtime, status và đường dẫn log. Nếu runtime biến động, chạy ít nhất ba lần và báo cáo median cùng khoảng biến thiên.

9. Chỉ số đánh giá

Nhóm chỉ số	Các chỉ số phải xem xét
Chất lượng	Recall@1/5/10/50, MRR, nDCG@k, P50/P95 latency
Hiệu năng/chi phí	QPS, Index build time, Index size, Proof success rate nếu tích hợp

Không phải mọi chỉ số đều dùng làm chỉ số chính. Nhóm phải chọn 1–2 primary metrics, các chỉ số còn lại dùng để giải thích trade-off và giới hạn.

10. Sản phẩm bàn giao

- Benchmark split
- Inverted index
- Search API/CLI
- BM25 baseline
- Hybrid reranker
- Evaluation scripts
- Error analysis
- README/demo

11. Tiêu chí nghiệm thu tối thiểu

- Dùng split cố định và cùng benchmark với Nhóm 4
- Có BM25 baseline
- Đo cả quality và latency
- Có ít nhất 3 kích thước index
- Có error analysis và index có thể tái tạo

12. Khả năng phát triển thành bài báo

Có thể phát triển thành bài về field-aware hoặc structure-aware premise retrieval, hoặc nghiên cứu ảnh hưởng của chất lượng truy hồi tới tỷ lệ chứng minh Lean.

Để đi tiếp theo hướng bài báo, nhóm phải xác định đóng góp mới so với công trình liên quan, bổ sung baseline mạnh, dùng split công bằng, chạy đủ lặp/ablation và trình bày threat to validity — các nguy cơ làm kết luận thiếu tin cậy.

NHÓM 4. Lean Premise Retrieval - Truy hồi dense và ANN

TÓM TẮT ĐỀ BÀI

Quy mô nhóm: 6 bạn. Xây hệ thống tìm premise bằng embedding và chỉ mục láng giềng gần đúng; tối ưu đánh đổi giữa recall, độ trễ và bộ nhớ. Nguồn khởi đầu: [3], [4], [5], [8], [9]; xem Phụ lục D.

1. Mục tiêu và câu hỏi nghiên cứu

Xây hệ thống tìm premise bằng embedding và chỉ mục láng giềng gần đúng; tối ưu đánh đổi giữa recall, độ trễ và bộ nhớ.

24. ANN giảm độ trễ bao nhiêu so với exact cosine trên cùng embedding?

25. HNSW/IVF và tham số tìm kiếm tạo Pareto frontier recall–latency–memory như thế nào?

Vai trò của Dữ liệu lớn: Exact search phải so truy vấn với mọi vector. Khi số premise và số truy vấn tăng, chi phí tính toán và bộ nhớ trở thành nút thắt; cần chỉ mục xấp xỉ và phục vụ tải.

2. Dữ liệu thực tế

- Cùng theorem–premise split với Nhóm 3.
- Statement, tên định lý, namespace và dependency từ LeanDojo Benchmark 4/mathlib4.
- Embedding phải được version hóa cùng tên model, pooling và kích thước vector.

YÊU CẦU DỮ LIỆU

Không tải và đưa toàn bộ dữ liệu lớn vào Git. Chỉ lưu sample hợp pháp, manifest, checksum và script tải/tạo dữ liệu. Mọi split phải cố định và được version hóa.

3. Kiến trúc gợi ý

- Batch encoding
- Vector store
- Exact cosine baseline
- Faiss Flat/HNSW/IVF
- Parameter sweep
- Search API
- Optional Lean evaluation

Nhóm có thể thay kiến trúc nếu giải thích được tác động tới tính đúng, chi phí, khả năng mở rộng và khả năng tích hợp.

4. Công cụ gợi ý

- Sentence Transformers hoặc encoder tương đương
- Faiss CPU/GPU
- PySpark cho batch embedding
- Parquet
- FastAPI/Locust hoặc công cụ load test

5. Phân vai

Vai trò	Trách nhiệm chính
Thành viên 1	Data contract

Vai trò	Trách nhiệm chính
Thành viên 2	Embedding pipeline
Thành viên 3	Exact baseline
Thành viên 4	ANN indexing
Thành viên 5	Serving/load test
Thành viên 6	Evaluation/Pareto analysis

Sáu vai trò tương ứng sáu người chính. Mỗi vai trò cần một người khác có thể chạy lại và giải thích khi bảo vệ.

6. Nhiệm vụ từng tuần

Tuần	Nhiệm vụ bắt buộc	Mình chứng cần nộp
1	Chốt benchmark chung với Nhóm 3; chọn 1 encoder chính và ngân sách tài nguyên.	Specification; model card sơ bộ; sample queries.
2	Tạo embedding sample và kiểm tra tương đồng; lưu vector + metadata.	10K vectors; schema; nearest-neighbor sanity checks.
3	Cài exact cosine/IndexFlat baseline.	Baseline Recall@k, latency, memory.
4	Cài HNSW và/hoặc IVF; xác định tham số cần quét.	Hai ANN configurations; build/search scripts.
5	Batch embedding bằng Spark; quản lý checkpoint và lỗi.	Pipeline 100K+; throughput và failure logs.
6	Xây search service; warm-up, cache và concurrency test.	API; load-test plan; reproducible container.
7	Chạy 10K, 100K, toàn index; sweep nprobe/efSearch.	Raw Pareto data: recall, P95, QPS, RAM, build time.
8	Ablation encoder/index; error analysis theo chủ đề.	Pareto plots; error taxonomy; limitation note.
9	So sánh sparse/dense/hybrid với Nhóm 3.	Common benchmark table; report draft.
10	Demo truy vấn và bảo vệ trade-off đã chọn.	Final index/service; report; demo; contribution log.

7. Baseline bắt buộc

Exact cosine search hoặc Faiss IndexFlat trên cùng embedding; kết quả chính xác nhưng chi phí tuyến tính theo số vector.

QUY TẮC BASELINE

Baseline phải dùng cùng dữ liệu, cùng split, cùng phần cứng và cùng timeout với phương pháp chính. Không thay đổi điều kiện để làm phương pháp mới trông tốt hơn.

8. Thí nghiệm khả năng mở rộng

- 10K → 100K → toàn index
- Flat vs HNSW vs IVF
- nprobe/efSearch thấp–trung bình–cao
- CPU vs GPU nếu có
- 1–50 client đồng thời
- Kích thước/vector dimension nếu đủ thời gian

Mỗi cấu hình cần lưu: timestamp, commit, data version, machine, software version, seed, parameters, runtime, status và đường dẫn log. Nếu runtime biến động, chạy ít nhất ba lần và báo cáo median cùng khoảng biến thiên.

9. Chỉ số đánh giá

Nhóm chỉ số	Các chỉ số phải xem xét
Chất lượng	Recall@k, MRR, P50/P95 latency, QPS
Hiệu năng/chí phí	Build time, Peak RAM/GPU memory, Index size, Proof success nếu tích hợp

Không phải mọi chỉ số đều dùng làm chỉ số chính. Nhóm phải chọn 1–2 primary metrics, các chỉ số còn lại dùng để giải thích trade-off và giới hạn.

10. Sản phẩm bàn giao

- Embedding pipeline
- Vector dataset manifest
- Exact baseline
- ANN indexes
- Search service
- Pareto plots
- Evaluation scripts
- README/demo

11. Tiêu chí nghiệm thu tối thiểu

- Cùng split với Nhóm 3
- Exact baseline và ít nhất một ANN index
- Có parameter sweep
- Báo cáo Pareto thay vì chỉ một cấu hình
- Index và embedding tái tạo được

12. Khả năng phát triển thành bài báo

Có thể phát triển thành bài về điều chỉnh ANN cho formal mathematics, hoặc kết hợp sparse–dense dưới ngân sách độ trễ/bộ nhớ xác định.

Để đi tiếp theo hướng bài báo, nhóm phải xác định đóng góp mới so với công trình liên quan, bổ sung baseline mạnh, dùng split công bằng, chạy đủ lặp/ablation và trình bày threat to validity — các nguy cơ làm kết luận thiếu tin cậy.

NHÓM 5. Autoformalization Pipeline - Chuyển đề toán sang Lean theo lô

TÓM TẮT ĐỀ BÀI

Quy mô nhóm: 6 bạn. Xây quy trình sinh hàng loạt phát biểu Lean từ đề toán tự nhiên, kiểm tra bằng Lean và đánh giá riêng tính hợp lệ hình thức với đúng ngữ nghĩa. Nguồn khởi đầu: [1], [2], [3], [4], [10]; xem Phụ lục D.

1. Mục tiêu và câu hỏi nghiên cứu

Xây quy trình sinh hàng loạt phát biểu Lean từ đề toán tự nhiên, kiểm tra bằng Lean và đánh giá riêng tính hợp lệ hình thức với đúng ngữ nghĩa.

33. Batching và số candidate ảnh hưởng thế nào tới throughput, chi phí và compile rate?
34. Làm sao phát hiện trường hợp Lean chấp nhận nhưng phát biểu hình thức sai ý đề tự nhiên?

Vai trò của Dữ liệu lớn: Hàng nghìn đến hàng chục nghìn bài tạo tải inference, hàng đợi kiểm chứng, nhiều worker Lean, log lớn và chi phí token/GPU; cần lập lịch, batch và chịu lỗi.

2. Dữ liệu thực tế

- ProofNet: benchmark 371 ví dụ, lưu ý bản gốc dùng Lean 3; cần xác định chiến lược Lean 4 hoặc dùng để đánh giá statement alignment.
- NuminaMath-CoT: lấy mẫu bài toán lớn để đo throughput; phần lớn không có formal label.
- Tập 100–300 mẫu semantic audit do nhóm xây và hai người kiểm tra độc lập.

YÊU CẦU DỮ LIỆU

Không tải và đưa toàn bộ dữ liệu lớn vào Git. Chỉ lưu sample hợp pháp, manifest, checksum và script tải/tạo dữ liệu. Mọi split phải cố định và được version hóa.

3. Kiến trúc gợi ý

35. Input queue
36. Prompt/version manager
37. Batch LLM inference
38. Candidate store
39. Lean worker pool
40. Result/error store
41. Semantic audit sample

Nhóm có thể thay kiến trúc nếu giải thích được tác động tới tính đúng, chi phí, khả năng mở rộng và khả năng tích hợp.

4. Công cụ gợi ý

- Lean 4
- LLM API hoặc mô hình mở
- Ray/Spark batch hoặc hàng đợi Python
- Docker
- Parquet/SQLite
- MLflow/W&B hoặc manifest tự xây

5. Phân vai

Vai trò	Trách nhiệm chính
---------	-------------------

Vai trò	Trách nhiệm chính
Thành viên 1	Data/evaluation
Thành viên 2	Prompt & model
Thành viên 3	Batch scheduler
Thành viên 4	Lean worker pool
Thành viên 5	Semantic audit
Thành viên 6	Metrics/reproducibility

Sáu vai trò tương ứng sáu người chính. Mỗi vai trò cần một người khác có thể chạy lại và giải thích khi bảo vệ.

6. Nhiệm vụ từng tuần

Tuần	Nhiệm vụ bắt buộc	Minh chứng cần nộp
1	Chốt phạm vi: statement autoformalization; chọn Lean version, model và ngân sách.	Specification; risk/ethics note; 20 examples.
2	Chuẩn hóa input/output; dựng Lean checker và semantic rubric.	Schema; checker script; audit guideline.
3	Zero-shot, one-request/one-candidate baseline.	Baseline compile/pass metrics; prompt v1.
4	Xây batch inference, queue và candidate store.	Pipeline 100 samples; retry/error logs.
5	Mở rộng Lean worker pool; timeout và sandbox.	Concurrency test; worker throughput; failure recovery.
6	Sinh nhiều candidate hoặc few-shot variant; version prompts.	Variants; token/cost logs; sample audit.
7	Chạy 100, 1K, 10K; thay batch size và worker count.	Throughput, P95, compile rate, cost; raw results.
8	Semantic audit và error taxonomy; ablation.	Agreement, semantic accuracy, failure examples.
9	Đóng gói pipeline, corpus và report.	Release candidate; model/data card; report.
10	Demo NL → Lean → checker; bảo vệ cá nhân.	Final package; demo; oral defense.

7. Baseline bắt buộc

Zero-shot; một bài trên mỗi request; sinh một candidate; Lean kiểm tra một worker; không repair.

QUY TẮC BASELINE

Baseline phải dùng cùng dữ liệu, cùng split, cùng phần cứng và cùng timeout với phương pháp chính. Không thay đổi điều kiện để làm phương pháp mới trông tốt hơn.

8. Thí nghiệm khả năng mở rộng

- 100 → 1K → 10K bài
- Batch size 1/8/32 hoặc phù hợp
- 1/2/4+ Lean workers
- 1 vs nhiều candidate
- Zero-shot vs few-shot
- Có/không retry

Mỗi cấu hình cần lưu: timestamp, commit, data version, machine, software version, seed, parameters, runtime, status và đường dẫn log. Nếu runtime biến động, chạy ít nhất ba lần và báo cáo median cùng khoảng biến thiên.

9. Chỉ số đánh giá

Nhóm chỉ số	Các chỉ số phải xem xét
Chất lượng	Parse rate, Compile rate, Semantic accuracy trên audit set, Pass@k
Hiệu năng/chi phí	Throughput, P50/P95 latency, Token/GPU cost, Failure rate theo loại

Không phải mọi chỉ số đều dùng làm chỉ số chính. Nhóm phải chọn 1–2 primary metrics, các chỉ số còn lại dùng để giải thích trade-off và giới hạn.

10. Sản phẩm bàn giao

- Versioned prompt set
- Batch pipeline
- Lean checker workers
- Formalized corpus + manifest
- Semantic audit set
- Error taxonomy
- Raw results
- README/demo

11. Tiêu chí nghiệm thu tối thiểu

- Phân biệt compile correctness và semantic correctness
- Có semantic audit hai người
- Có zero-shot baseline
- Có 3 mức quy mô
- Ghi token/cost và failure logs
- Không tuyên bố đúng ngữ nghĩa chỉ dựa vào Lean compile

12. Khả năng phát triển thành bài báo

Có thể phát triển thành bài về autoformalization có khả năng mở rộng, hoặc nghiên cứu quan hệ giữa batching, candidate diversity, compile rate, semantic accuracy và chi phí.

Để đi tiếp theo hướng bài báo, nhóm phải xác định đóng góp mới so với công trình liên quan, bổ sung baseline mạnh, dùng split công bằng, chạy đủ lặp/ablation và trình bày threat to validity — các nguy cơ làm kết luận thiếu tin cậy.

NHÓM 6. Verifier-Guided Repair - Sửa bản Lean dựa trên phản hồi kiểm chứng

TÓM TẮT ĐỀ BÀI

Quy mô nhóm: 6 bạn. Phân loại lỗi Lean, định tuyến chiến lược sửa và xây vòng lặp sửa–kiểm tra có giới hạn chi phí, thời gian và số lần thử. Nguồn khởi đầu: [2], [3], [4], [10]; xem Phụ lục D.

1. Mục tiêu và câu hỏi nghiên cứu

Phân loại lỗi Lean, định tuyến chiến lược sửa và xây vòng lặp sửa–kiểm tra có giới hạn chi phí, thời gian và số lần thử.

42. Thông báo lỗi có cấu trúc cải thiện repair success so với retry nguyên prompt không?
43. Chính sách retry/worker nào tối ưu compile rate dưới ngân sách token và thời gian cố định?

Vai trò của Dữ liệu lớn: Khi số candidate tăng, hệ thống phải xử lý hàng đợi lỗi, nhiều worker, retry và log; nếu lập lịch kém, tail latency và chi phí tăng mạnh.

2. Dữ liệu thực tế

- Candidate lỗi + compiler messages từ Nhóm 5.
- Bộ lỗi tổng hợp có nhãn: syntax, unknown identifier, type mismatch, missing import, timeout, semantic mismatch.
- Một tập hold-out không dùng để thiết kế rule nhằm đánh giá công bằng.

YÊU CẦU DỮ LIỆU

Không tải và đưa toàn bộ dữ liệu lớn vào Git. Chỉ lưu sample hợp pháp, manifest, checksum và script tải/tạo dữ liệu. Mọi split phải cố định và được version hóa.

3. Kiến trúc gợi ý

44. Lean verifier
45. Structured error parser
46. Error taxonomy
47. Router
48. Rule/LLM repair
49. Bounded retry loop
50. Queue and result store

Nhóm có thể thay kiến trúc nếu giải thích được tác động tới tính đúng, chi phí, khả năng mở rộng và khả năng tích hợp.

4. Công cụ gợi ý

- Lean 4
- Python
- Redis/RabbitMQ hoặc queue đơn giản
- LLM API/model mở
- Docker
- Prometheus/logging (không bắt buộc)

5. Phân vai

Vai trò	Trách nhiệm chính
Thành viên 1	Error dataset

Vai trò	Trách nhiệm chính
Thành viên 2	Parser/taxonomy
Thành viên 3	Rule repair
Thành viên 4	LLM repair
Thành viên 5	Queue/worker system
Thành viên 6	Evaluation/cost analysis

Sáu vai trò tương ứng sáu người chính. Mỗi vai trò cần một người khác có thể chạy lại và giải thích khi bảo vệ.

6. Nhiệm vụ từng tuần

Tuần	Nhiệm vụ bắt buộc	Minh chứng cần nộp
1	Chốt error taxonomy, ngân sách retry và tiêu chí semantic safety.	Taxonomy v0.1; 50 labeled errors; baseline plan.
2	Nhận data contract từ Nhóm 5; dựng replayable error store.	Sample store; replay script; unit tests.
3	Baseline retry nguyên prompt một lần.	Repair success, compile rate, cost baseline.
4	Cài error parser và rule-based fixes cho lỗi đơn giản.	Parser accuracy; rule coverage; examples.
5	Cài router và LLM repair dùng error context.	Repair service v0.1; prompt versions.
6	Xây bounded loop, timeout, dedup retry và worker queue.	End-to-end service; recovery tests.
7	Scale candidates, workers và retry budget.	Queue latency, throughput, cost, success curves.
8	Ablation parser/router/context; semantic audit.	Ablation table; failure taxonomy; safety note.
9	Đóng gói service, benchmark và report.	Release candidate; README; raw logs; report.
10	Demo lỗi → phân loại → sửa → Lean kiểm tra.	Final service; demo; contribution log.

7. Baseline bắt buộc

Retry nguyên prompt đúng một lần, không phân loại lỗi và không đưa thông báo Lean có cấu trúc vào chiến lược sửa.

QUY TẮC BASELINE

Baseline phải dùng cùng dữ liệu, cùng split, cùng phần cứng và cùng timeout với phương pháp chính. Không thay đổi điều kiện để làm phương pháp mới trông tốt hơn.

8. Thí nghiệm khả năng mở rộng

- 1K → 10K candidates

- 1/2/4+ workers
- Retry budget 0/1/3
- Rule-only vs LLM-only vs hybrid
- Có/không structured error
- FIFO vs priority routing nếu đủ thời gian

Mỗi cấu hình cần lưu: timestamp, commit, data version, machine, software version, seed, parameters, runtime, status và đường dẫn log. Nếu runtime biến động, chạy ít nhất ba lần và báo cáo median cùng khoảng biến thiên.

9. Chỉ số đánh giá

Nhóm chỉ số	Các chỉ số phải xem xét
Chất lượng	Repair success, Final compile rate, Semantic pass trên audit, Attempts/sample
Hiệu năng/chí phí	Throughput, Queue P95, Wall time, Token cost, Error coverage

Không phải mọi chỉ số đều dùng làm chỉ số chính. Nhóm phải chọn 1–2 primary metrics, các chỉ số còn lại dùng để giải thích trade-off và giới hạn.

10. Sản phẩm bàn giao

- Labeled error set
- Parser + taxonomy
- Repair router/service
- Worker queue
- Replayable logs
- Cost–quality curves
- Test suite
- README/demo

11. Tiêu chí nghiệm thu tối thiểu

- Có hold-out error set
- So sánh với retry baseline
- Vòng lặp có giới hạn rõ
- Đo cả compile, semantic và cost
- Log có thể replay
- Không lặp vô hạn khi lỗi không sửa được

12. Khả năng phát triển thành bài báo

Có thể phát triển thành bài về error-aware repair policy, cost-aware retry hoặc scheduling của verifier workers dưới ngân sách giới hạn.

Để đi tiếp theo hướng bài báo, nhóm phải xác định đóng góp mới so với công trình liên quan, bổ sung baseline mạnh, dùng split công bằng, chạy đủ lặp/ablation và trình bày threat to validity — các nguy cơ làm kết luận thiếu tin cậy.

NHÓM 7. Proof-Search Analytics - Phân tích vết tìm kiếm chứng minh

TÓM TẮT ĐỀ BÀI

Quy mô nhóm: 6 bạn. Thu thập và phân tích proof state, tactic và search trace để mô tả hành vi tìm kiếm, dự đoán tactic hoặc cắt nhánh ít triển vọng. Nguồn khởi đầu: [3], [4], [5], [7]; xem Phụ lục D.

1. Mục tiêu và câu hỏi nghiên cứu

Thu thập và phân tích proof state, tactic và search trace để mô tả hành vi tìm kiếm, dự đoán tactic hoặc cắt nhánh ít triển vọng.

51. Đặc trưng nào dự đoán tactic tiếp theo tốt nhất trên các miền toán khác nhau?

52. Pruning giảm states expanded và thời gian bao nhiêu trước khi proof success rate giảm không chấp nhận được?

Vai trò của Dữ liệu lớn: Một theorem có thể sinh hàng trăm hoặc hàng nghìn states/events; toàn thư viện tạo log lớn, cần schema sự kiện, lưu theo cột, aggregation phân tán và replay.

2. Dữ liệu thực tế

- LeanDojo proof states, tactics, premises và traces; ưu tiên Benchmark 4 hoặc phiên bản hiện hành.
- Traces do nhóm tạo từ một prover/heuristic search cố định.
- Metadata theorem: chủ đề, độ dài, imports, search budget và kết quả.

YÊU CẦU DỮ LIỆU

Không tải và đưa toàn bộ dữ liệu lớn vào Git. Chỉ lưu sample hợp pháp, manifest, checksum và script tải/tạo dữ liệu. Mọi split phải cố định và được version hóa.

3. Kiến trúc gợi ý

- Trace collector
- Event schema
- Parquet trace lake
- Spark aggregation
- Feature extraction
- Tactic predictor/pruning policy
- Replay/evaluation

Nhóm có thể thay kiến trúc nếu giải thích được tác động tới tính đúng, chi phí, khả năng mở rộng và khả năng tích hợp.

4. Công cụ gợi ý

- LeanDojo v2/current
- Lean 4
- PySpark
- Parquet
- scikit-learn/PyTorch
- Streamlit/Superset hoặc notebook dashboard

5. Phân vai

Vai trò	Trách nhiệm chính
---------	-------------------

Vai trò	Trách nhiệm chính
Thành viên 1	Trace generation
Thành viên 2	Event schema/storage
Thành viên 3	Spark analytics
Thành viên 4	Tactic baseline/model
Thành viên 5	Pruning/replay
Thành viên 6	Evaluation/dashboard

Sáu vai trò tương ứng sáu người chính. Mỗi vai trò cần một người khác có thể chạy lại và giải thích khi bảo vệ.

6. Nhiệm vụ từng tuần

Tuần	Nhiệm vụ bắt buộc	Minh chứng cần nộp
1	Chốt đơn vị event, trace schema và search budget.	Schema v0.1; 5 traces minh họa; questions.
2	Thu 100–500 traces; kiểm tra completeness và timestamp.	Parquet sample; validation report.
3	Baseline: tactic frequency + search không pruning.	Baseline top-k accuracy và proof/search metrics.
4	Spark aggregation: branching, depth, failure, tactic transitions.	Feature tables; analytics notebook.
5	Tactic predictor hoặc heuristic branch score.	Model v0.1; fixed split; confusion/error analysis.
6	Cài pruning policy và replay evaluator.	Reproducible replay; no-pruning comparison.
7	Scale theorem count $\times$ search budget; lưu raw traces/results.	Throughput, storage, states, time, proof rate.
8	Pareto analysis và ablation theo feature/domain.	Pareto plot; ablation; domain slices.
9	Dashboard, report và đóng gói trace schema.	Release candidate; data card; report.
10	Demo trace $\rightarrow$ dashboard $\rightarrow$ pruning comparison.	Final package; demo; oral defense.

7. Baseline bắt buộc

Most-frequent tactic hoặc heuristic đơn giản; proof search không pruning với cùng timeout và search budget.

QUY TẮC BASELINE

Baseline phải dùng cùng dữ liệu, cùng split, cùng phần cứng và cùng timeout với phương pháp chính. Không thay đổi điều kiện để làm phương pháp mới trông tốt hơn.

8. Thí nghiệm khả năng mở rộng

- 100 → 1K → 10K theorems/traces tùy tài nguyên
- Search budget nhỏ/trung bình/lớn
- Không pruning vs nhiều threshold
- Feature ablation
- Theo miền toán/độ dài theorem

Mỗi cấu hình cần lưu: timestamp, commit, data version, machine, software version, seed, parameters, runtime, status và đường dẫn log. Nếu runtime biến động, chạy ít nhất ba lần và báo cáo median cùng khoảng biến thiên.

9. Chỉ số đánh giá

Nhóm chỉ số	Các chỉ số phải xem xét
Chất lượng	Events/s, Storage size, Aggregation runtime, Top-k tactic accuracy
Hiệu năng/chi phí	States expanded, Search wall time, Proof success rate, Timeout rate

Không phải mọi chỉ số đều dùng làm chỉ số chính. Nhóm phải chọn 1–2 primary metrics, các chỉ số còn lại dùng để giải thích trade-off và giới hạn.

10. Sản phẩm bàn giao

- Trace schema + data card
- Trace lake sample
- Spark analytics
- Baseline predictor
- Replay/pruning benchmark
- Dashboard
- Pareto/error analysis
- README/demo

11. Tiêu chí nghiệm thu tối thiểu

- Trace schema đầy đủ và version hóa
- So sánh cùng timeout/search budget
- Có baseline không pruning
- Báo cáo cả states và proof rate
- Có replay hoặc quy trình tái tạo
- Không kết luận nhanh hơn nếu proof rate giảm mà không phân tích

12. Khả năng phát triển thành bài báo

Có thể phát triển thành bài characterization về proof search, learned tactic prediction hoặc learned pruning theo miền toán và ngân sách tìm kiếm.

Để đi tiếp theo hướng bài báo, nhóm phải xác định đóng góp mới so với công trình liên quan, bổ sung baseline mạnh, dùng split công bằng, chạy đủ lặp/ablation và trình bày threat to validity — các nguy cơ làm kết luận thiếu tin cậy.

NHÓM 8. AI Math Tutor - Phân tích luồng học tập và phản hồi toán có kiểm chứng

TÓM TẮT ĐỀ BÀI

Quy mô nhóm: 6 bạn. Xử lý luồng sự kiện học tập, cập nhật hồ sơ theo thời gian thực và đưa ra gợi ý toán có kiểm chứng, đồng thời bảo vệ quyền riêng tư. Nguồn khởi đầu: [1], [11], [12], [13]; xem Phụ lục D.

1. Mục tiêu và câu hỏi nghiên cứu

Xử lý luồng sự kiện học tập, cập nhật hồ sơ theo thời gian thực và đưa ra gợi ý toán có kiểm chứng, đồng thời bảo vệ quyền riêng tư.

60. Streaming profile giảm độ trễ phản hồi bao nhiêu so với batch theo ngày?

61. Làm sao đánh giá đồng thời độ đúng toán học, độ trễ và giá trị sự phạm của gợi ý?

Vai trò của Dữ liệu lớn: Dữ liệu đến liên tục, nhiều người dùng, có sự kiện muộn/trùng, trạng thái dài hạn và yêu cầu P95 latency thấp; cần partition, stateful processing và kiểm thử chịu tải.

2. Dữ liệu thực tế

- Synthetic learner events: open_problem, submit_step, wrong_answer, request_hint, complete.
- Kho bài toán và lời giải có kiểm chứng; có thể lấy một tập nhỏ từ NuminaMath rồi gán đáp án/rule kiểm tra.
- Nếu dùng dữ liệu thật: phải ẩn danh, tối thiểu hóa trường và có phê duyệt phù hợp; bản mặc định của môn nên dùng dữ liệu mô phỏng.

YÊU CẦU DỮ LIỆU

Không tải và đưa toàn bộ dữ liệu lớn vào Git. Chỉ lưu sample hợp pháp, manifest, checksum và script tải/tạo dữ liệu. Mọi split phải cố định và được version hóa.

3. Kiến trúc gợi ý

62. Event generator
63. Kafka topics
64. Spark Structured Streaming
65. Stateful learner profile
66. Verified hint service
67. Parquet/Delta sink
68. Dashboard/alerts

Nhóm có thể thay kiến trúc nếu giải thích được tác động tới tính đúng, chi phí, khả năng mở rộng và khả năng tích hợp.

4. Công cụ gợi ý

- Apache Kafka
- Spark Structured Streaming
- PySpark
- Parquet/Delta
- Docker Compose
- Streamlit/Grafana
- SymPy/Lean/rule checker tùy nội dung

5. Phân vai

Vai trò	Trách nhiệm chính
Thành viên 1	Event/data/privacy
Thành viên 2	Kafka ingestion
Thành viên 3	Streaming/state
Thành viên 4	Hint/checker
Thành viên 5	Load test/monitoring
Thành viên 6	Evaluation/dashboard

Sáu vai trò tương ứng sáu người chính. Mỗi vai trò cần một người khác có thể chạy lại và giải thích khi bảo vệ.

6. Nhiệm vụ từng tuần

Tuần	Nhiệm vụ bắt buộc	Minh chứng cần nộp
1	Định nghĩa event schema, privacy constraints và learning scenario.	Schema; threat/privacy note; 20 event examples.
2	Xây event generator và batch baseline.	Replayable event files; daily-profile baseline.
3	Dựng Kafka và consumer; xử lý duplicate/event ID.	Kafka demo; ingestion rate; failure tests.
4	Structured Streaming và learner state cơ bản.	State update pipeline; checkpoint/restart test.
5	Xử lý late events/watermark; exactly-once/idempotency logic.	Correctness test; late-event report.
6	Xây verified hint service và dashboard.	Hint baseline; checker; dashboard v0.1.
7	Load test 100, 1K, 10K learners và nhiều event rates.	Throughput, P50/P95, state size, error rate.
8	Đánh giá hint validity, pedagogical rubric và fairness slices.	Human/rule audit; subgroup results; limitations.
9	Đóng gói Docker Compose, report và demo scenario.	Release candidate; README; report; video draft.
10	Demo live stream, learner state, alert và verified hint.	Final package; privacy statement; oral defense.

7. Baseline bắt buộc

Xử lý batch một lần mỗi ngày, cập nhật hồ sơ bằng rule đơn giản và sinh gợi ý theo mẫu; không xử lý luồng thời gian thực.

QUY TẮC BASELINE

Baseline phải dùng cùng dữ liệu, cùng split, cùng phần cứng và cùng timeout với phương pháp chính. Không thay đổi điều kiện để làm phương pháp mới trông tốt hơn.

8. Thí nghiệm khả năng mở rộng

- 100 → 1K → 10K learners
- 100/1K/10K events per second tùy máy
- 1/3/6 Kafka partitions
- Batch vs streaming
- Có/không late-event handling
- Một vs nhiều concurrent hint requests

Mỗi cấu hình cần lưu: timestamp, commit, data version, machine, software version, seed, parameters, runtime, status và đường dẫn log. Nếu runtime biến động, chạy ít nhất ba lần và báo cáo median cùng khoảng biến thiên.

9. Chỉ số đánh giá

Nhóm chỉ số	Các chỉ số phải xem xét
Chất lượng	Throughput, P50/P95 end-to-end latency, Processing lag, Duplicate/lost-event rate
Hiệu năng/chi phí	State size, Late-event correctness, Hint validity, Pedagogical usefulness rubric, Fairness slices

Không phải mọi chỉ số đều dùng làm chỉ số chính. Nhóm phải chọn 1–2 primary metrics, các chỉ số còn lại dùng để giải thích trade-off và giới hạn.

10. Sản phẩm bàn giao

- Event schema/generator
- Kafka + streaming pipeline
- Learner state store
- Verified hint service
- Load-test scripts
- Dashboard
- Privacy/data card
- Raw results
- README/demo

11. Tiêu chí nghiệm thu tối thiểu

- Dùng dữ liệu mô phỏng hoặc đã ẩn danh
- Có batch baseline
- Có checkpoint/restart và late-event test
- Có 3 mức tải
- Đo P95 chứ không chỉ average
- Đánh giá hint đúng và có ích, không chỉ phản hồi nhanh

12. Khả năng phát triển thành bài báo

Có thể phát triển thành bài về learner modeling thời gian thực, verified feedback, hoặc trade-off giữa latency, mathematical correctness và pedagogical usefulness.

Để đi tiếp theo hướng bài báo, nhóm phải xác định đóng góp mới so với công trình liên quan, bổ sung baseline mạnh, dùng split công bằng, chạy đủ lặp/ablation và trình bày threat to validity — các nguy cơ làm kết luận thiếu tin cậy.

Phụ lục A. Mẫu đề cương 1 trang - nộp tuần 1

Tên nhóm và đề tài:

Câu hỏi nghiên cứu:

Nút thắt Big Data:

Dữ liệu và giấy phép:

Baseline:

Ba mức quy mô/tải:

Primary metrics:

Kiến trúc bản nháp:

Phân vai:

Rủi ro và phương án dự phòng:

Phụ lục B. Mẫu ghi một lần chạy

Trường	Nội dung cần ghi
run_id	Mã duy nhất của lần chạy
timestamp	Thời điểm bắt đầu
git_commit	Commit mã nguồn
data_version	Snapshot/manifest dữ liệu
machine	CPU, RAM, GPU, hệ điều hành
software	Phiên bản Python/Lean/Spark/Kafka/Faiss liên quan
parameters	Toàn bộ tham số chính
seed/timeout	Seed và giới hạn thời gian
metrics	Kết quả chất lượng và chi phí
status/log	Thành công/thất bại và đường dẫn log

Phụ lục C. Cấu trúc báo cáo 12-20 trang

69. Tóm tắt: bài toán, phương pháp, dữ liệu và kết quả chính.
70. 1. Giới thiệu: động cơ, câu hỏi và vai trò Big Data.
71. 2. Công trình liên quan: 3–8 nguồn gần nhất và khoảng trống.
72. 3. Dữ liệu: nguồn, giấy phép, schema, split, quality và limitations.
73. 4. Phương pháp/kiến trúc: pipeline, thuật toán và độ phức tạp.
74. 5. Thiết lập thí nghiệm: baseline, scale, metric, machine, seed, lặp.
75. 6. Kết quả: bảng, biểu đồ scalability và trade-off.
76. 7. Ablation, error analysis và threat to validity.
77. 8. Kết luận, sản phẩm và hướng phát triển.
78. Phụ lục: AI-use statement, contribution statement và lệnh tái lập.

Phụ lục D. Danh mục nguồn chung

- [1] **AI-MO/NuminaMath-CoT**. <https://huggingface.co/datasets/AI-MO/NuminaMath-CoT> — Dataset card; khoảng 860K bài toán và lời giải CoT tại thời điểm kiểm tra.
- [2] **ProofNet repository**. <https://github.com/zhangir-azerbayev/ProofNet> — Benchmark 371 ví dụ autoformalization/formal proving; bản gốc dùng Lean 3.
- [3] **LeanDojo documentation**. <https://leandojo.readthedocs.io/> — Tài liệu và liên kết tới LeanDojo Benchmark 4.
- [4] **LeanDojo website**. <https://leandojo.org/leandojo.html> — Mô tả toolkit và benchmark theorem proving/premise retrieval.
- [5] **mathlib4**. <https://github.com/leanprover-community/mathlib4> — Thư viện toán học của Lean 4.
- [6] **Spark MinHashLSH**. <https://spark.apache.org/docs/latest/ml-features.html#locality-sensitive-hashing> — Tài liệu LSH trong Spark ML.
- [7] **Apache Spark**. <https://spark.apache.org/docs/latest/> — Tài liệu xử lý dữ liệu phân tán và PySpark.
- [8] **Faiss documentation**. <https://faiss.ai/index.html> — Thư viện tìm kiếm tương đồng và phân cụm vector.
- [9] **Faiss repository**. <https://github.com/facebookresearch/faiss> — Mã nguồn, tutorial và tài liệu chỉ mục.
- [10] **ProofNet paper**. <https://arxiv.org/abs/2302.12433> — Bài báo giới thiệu benchmark autoformalization.
- [11] **Apache Kafka documentation**. <https://kafka.apache.org/documentation/> — Nền tảng sự kiện phân tán.
- [12] **Spark Structured Streaming**. <https://spark.apache.org/docs/latest/streaming/index.html> — Tài liệu xử lý luồng có cấu trúc.
- [13] **Spark-Kafka integration**. <https://spark.apache.org/docs/latest/structured-streaming-kafka-integration.html> — Hướng dẫn tích hợp Kafka với Structured Streaming.